DATA VISUALIZATION

Data visualization is the process of presenting information through charts and graphs so that patterns, comparisons, and changes can be understood quickly.

Python provides several useful visualization libraries, including Matplotlib, Seaborn, and Plotly. In this document, we will explore Matplotlib and Seaborn with new example data.

MATPLOTLIB

Matplotlib is a popular Python library for creating visualizations. It gives detailed control over figures, axes, labels, markers, and other graphical elements.

A Matplotlib figure can contain the following main components:

- Figure
- Axes
- Axis
- Artists

FIGURE

A Figure is the overall container for one or more plots. Multiple plots can be arranged inside the same Figure.

AXES

Axes represent the actual plotting area. Labels can be added with set_xlabel() and set_ylabel().

AXIS

An Axis manages values, tick marks, and the visible limits used on a plot.

ARTISTS

Everything visible inside a Matplotlib figure, such as lines, text, markers, and shapes, is treated as an Artist.

PYPLOT

pyplot is the commonly used Matplotlib interface. It provides functions for creating line charts, bar charts, histograms, scatter plots, pie charts, and area charts.

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

LINE CHART

A line chart connects data points with lines and is useful for observing changes across an ordered sequence.

Here, the month names form the x-axis and the sales values form the y-axis. The marker makes each observation easy to identify.

BAR GRAPH

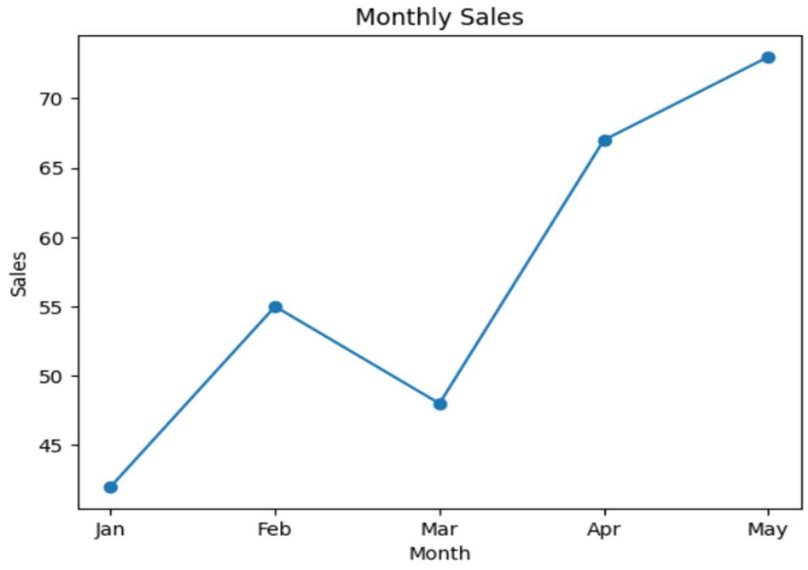
A bar graph compares values across separate categories. The height of each rectangle represents the value of that category.

Code Snippet

```
months = np.array(["Jan", "Feb", "Mar", "Apr", "May"])
sales = np.array([42, 55, 48, 67, 73])

plt.plot(months, sales, marker="o")
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```

Output



Explanation

The bar() function creates the bars. Different categories are placed along the x-axis and their quantities are shown on the y-axis.

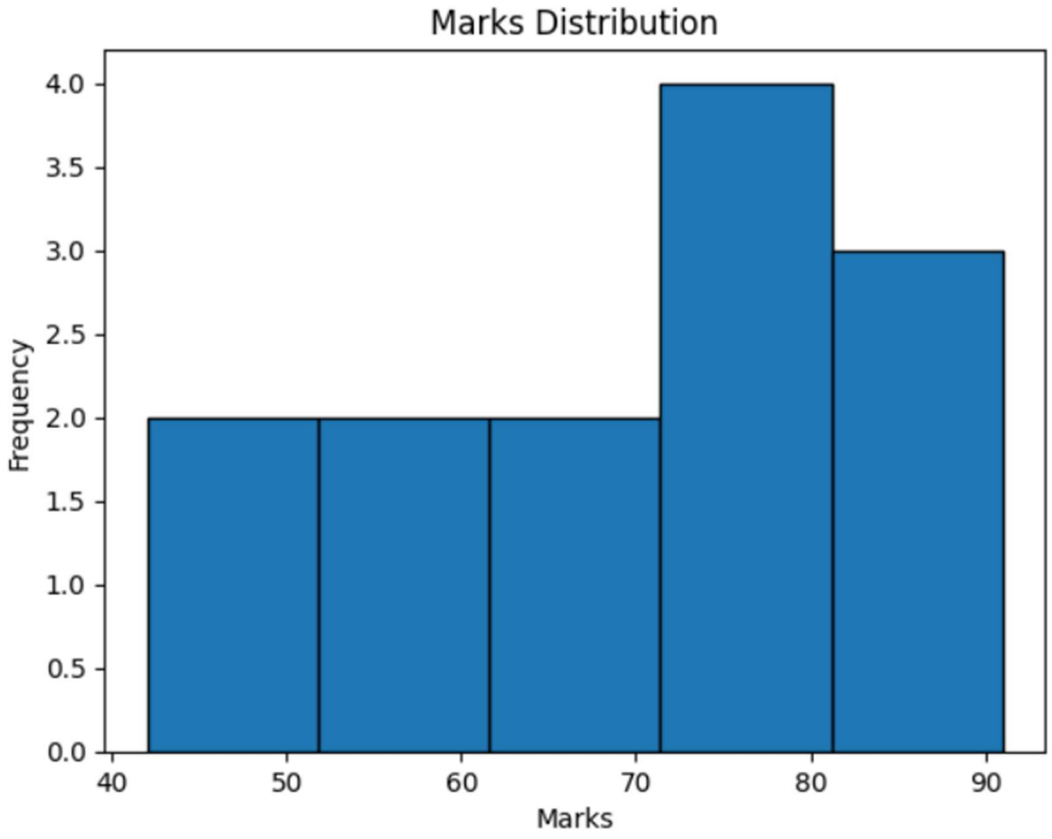
HISTOGRAM

A histogram groups numerical observations into intervals and shows how frequently values occur in each interval.

```
marks = np.array([42, 51, 55, 61, 64, 66, 72, 75, 78, 81, 84, 89, 91])

plt.hist(marks, bins=5, edgecolor="black")
plt.title("Marks Distribution")
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.show()
```

Output



Explanation

The bins parameter controls the number of intervals. The histogram helps identify where most observations are concentrated.

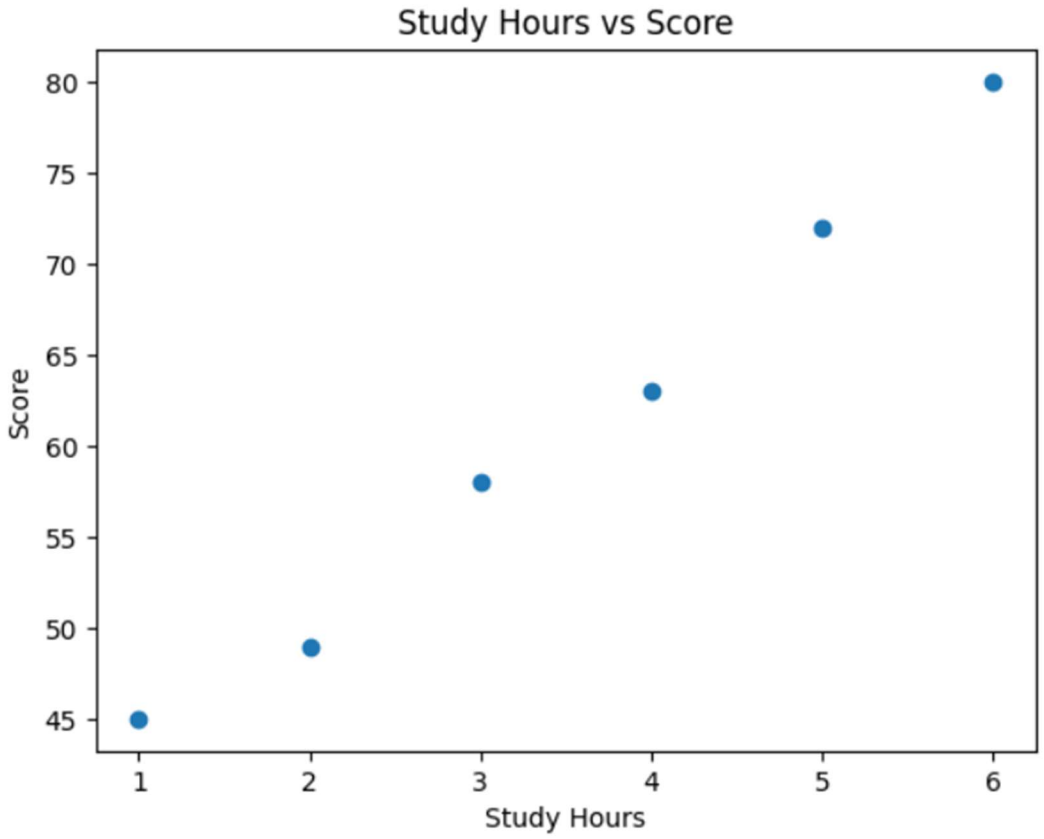
SCATTER PLOT

A scatter plot uses individual points to show the relationship between two numerical variables.

```
study_hours = [1, 2, 3, 4, 5, 6]
scores = [45, 49, 58, 63, 72, 80]

plt.scatter(study_hours, scores)
plt.title("Study Hours vs Score")
plt.xlabel("Study Hours")
plt.ylabel("Score")
plt.show()
```

Output



Explanation

Each point represents one observation. A visible upward pattern can suggest that the two variables are positively related.

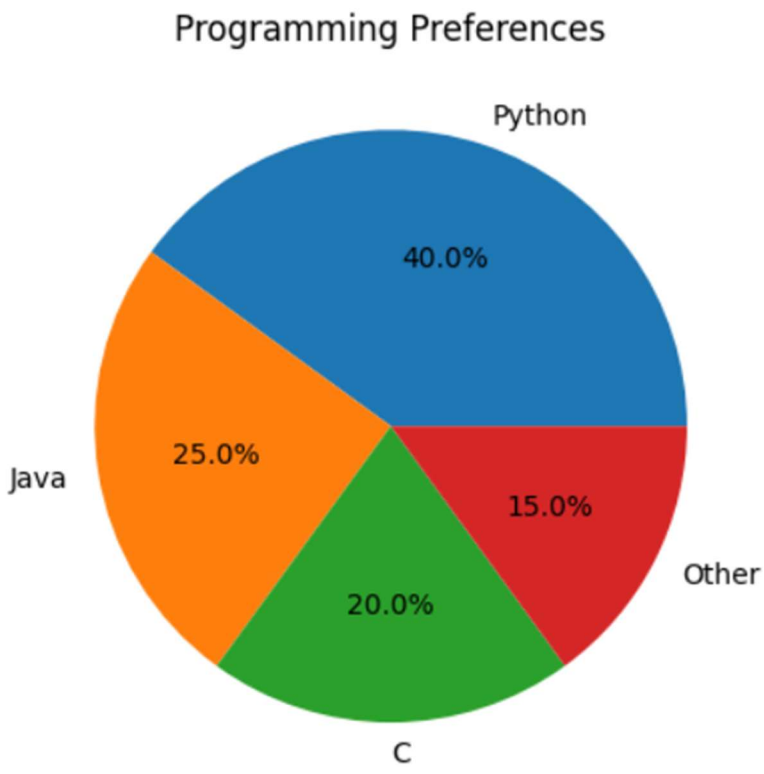
PIE CHART

A pie chart divides a complete set into proportional sections. It is useful when the parts of a whole need to be compared.

```
labels = ["Python", "Java", "C", "Other"]
values = [40, 25, 20, 15]

plt.pie(values, labels=labels, autopct="%1.1f%%")
plt.title("Programming Preferences")
plt.show()
```

Output



Explanation

The values determine the size of each section, while labels identify the categories.

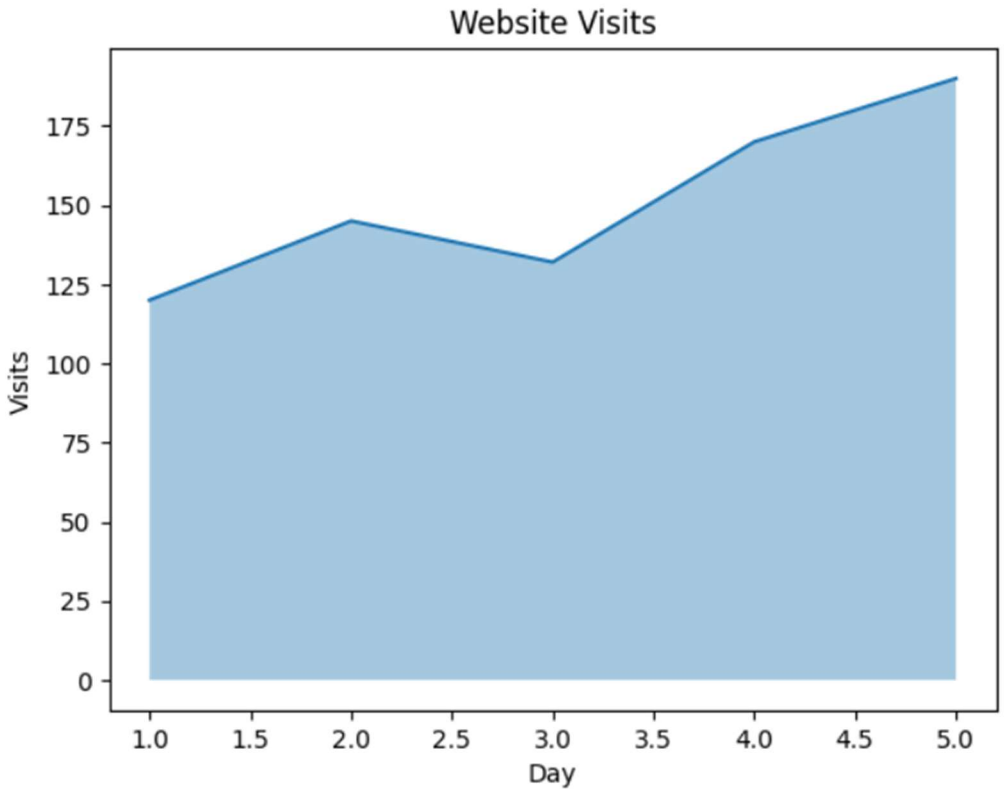
AREA CHART

An area chart highlights the space between a data line and the x-axis. It can be created using fill_between() or stackplot().

```
days = [1, 2, 3, 4, 5]
visits = [120, 145, 132, 170, 190]

plt.fill_between(days, visits, alpha=0.4)
plt.plot(days, visits)
plt.title("Website Visits")
plt.xlabel("Day")
plt.ylabel("Visits")
plt.show()
```

Output



Explanation

fill_ between() fills the area under the plotted values, making changes in the magnitude easier to notice.

SEABORN

Seaborn is a Python visualization library built on Matplotlib. It provides convenient functions, statistical plots, themes, and color palettes.

Common Seaborn plot categories include:

- Relational: scatterplot(), lineplot(), relplot()
- Categorical: barplot(), countplot(), boxplot(), violinplot(), pointplot(), catplot()
- Distribution: histplot(), kdeplot(), rugplot()
- Regression: regplot(), lmplot()
- Matrix: heatmap(), clustermap()

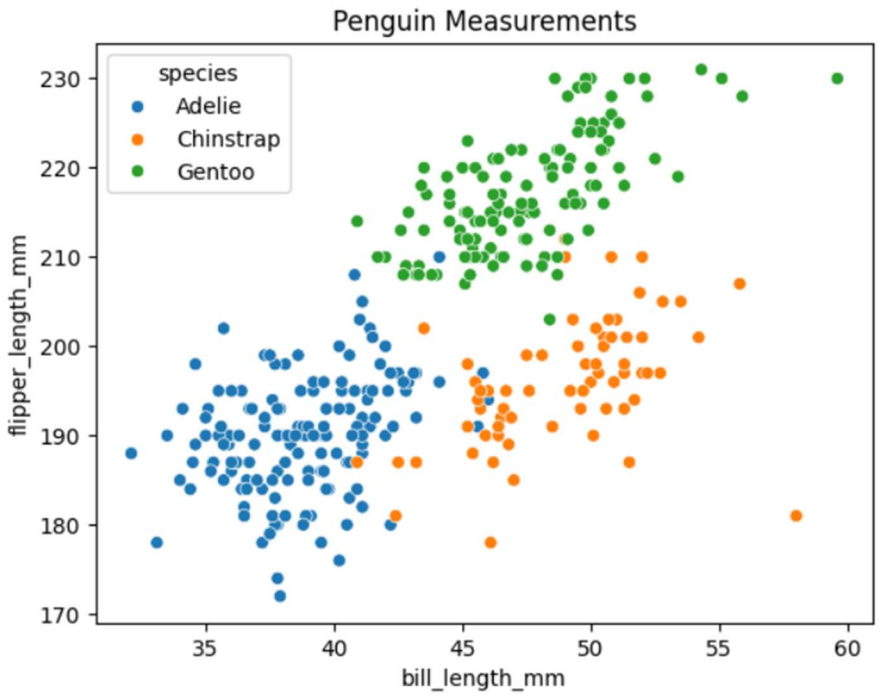
```
import seaborn as sns
import matplotlib.pyplot as plt
```

SEABORN SCATTER PLOT

A scatter plot in Seaborn can use hue to separate observations into groups.

```
data = sns.load_dataset("penguins")
sns.scatterplot(data=data, x="bill_length_mm", y="flipper_length_mm", hue="species")
plt.title("Penguin Measurements")
plt.show()
```

Output



Explanation

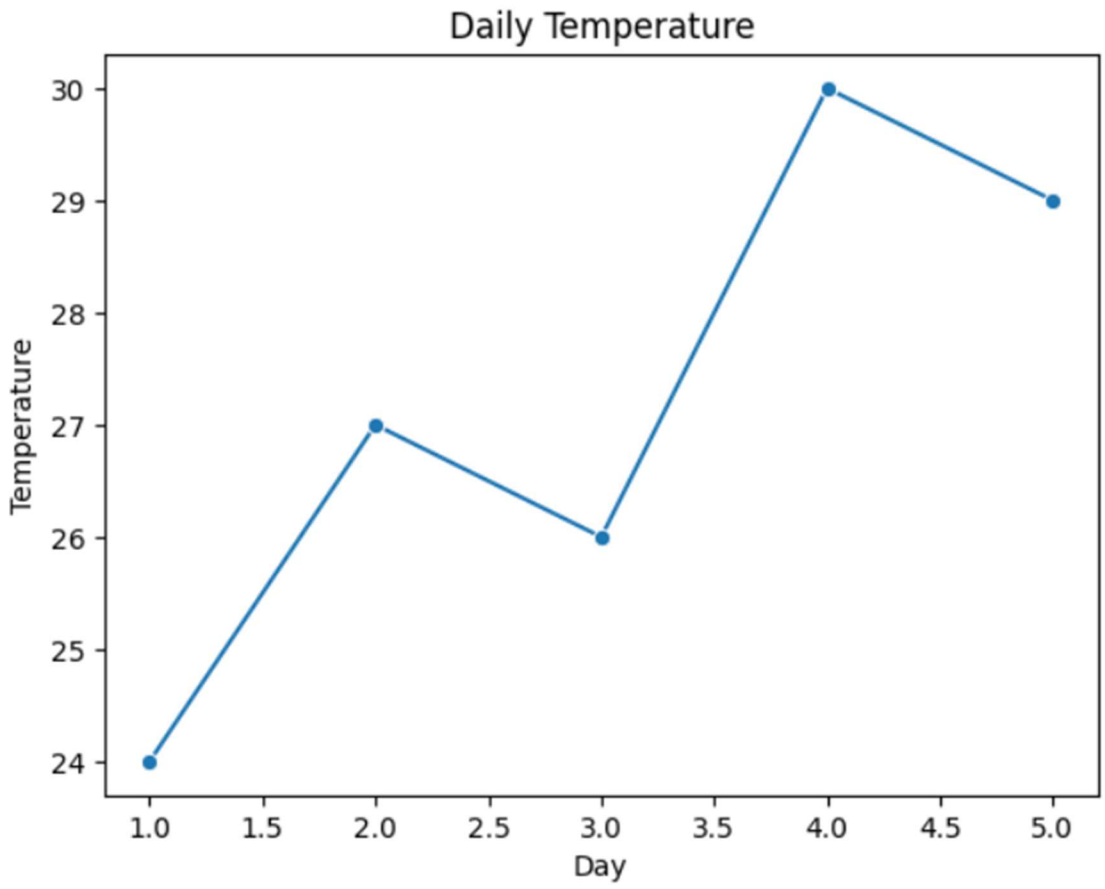
The hue parameter assigns different visual groups according to the selected column.

SEABORN LINE PLOT

```
days = [1, 2, 3, 4, 5]
temperature = [24, 27, 26, 30, 29]

sns.lineplot(x=days, y=temperature, marker="o")
plt.title("Daily Temperature")
plt.xlabel("Day")
plt.ylabel("Temperature")
plt.show()
```

Output



Explanation

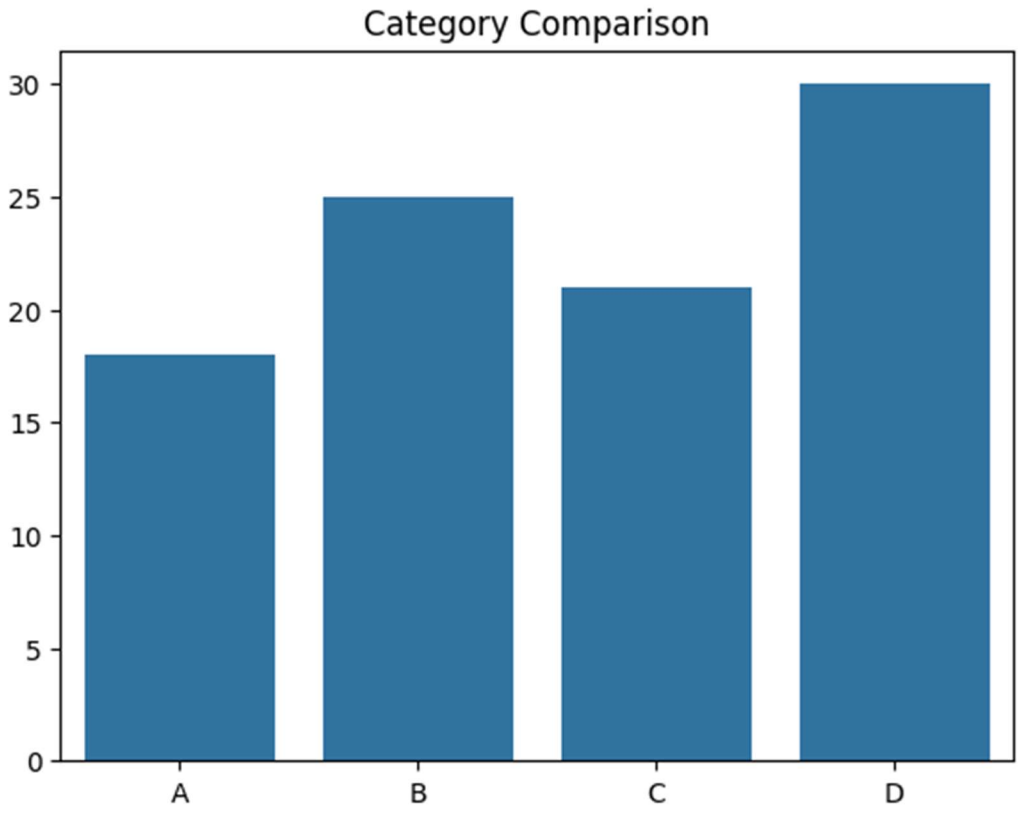
A line plot is useful for showing how a numerical measurement changes across an ordered variable.

SEABORN BAR PLOT

```
categories = ["A", "B", "C", "D"]
values = [18, 25, 21, 30]

sns.barplot(x=categories, y=values)
plt.title("Category Comparison")
plt.show()
```

Output



Explanation

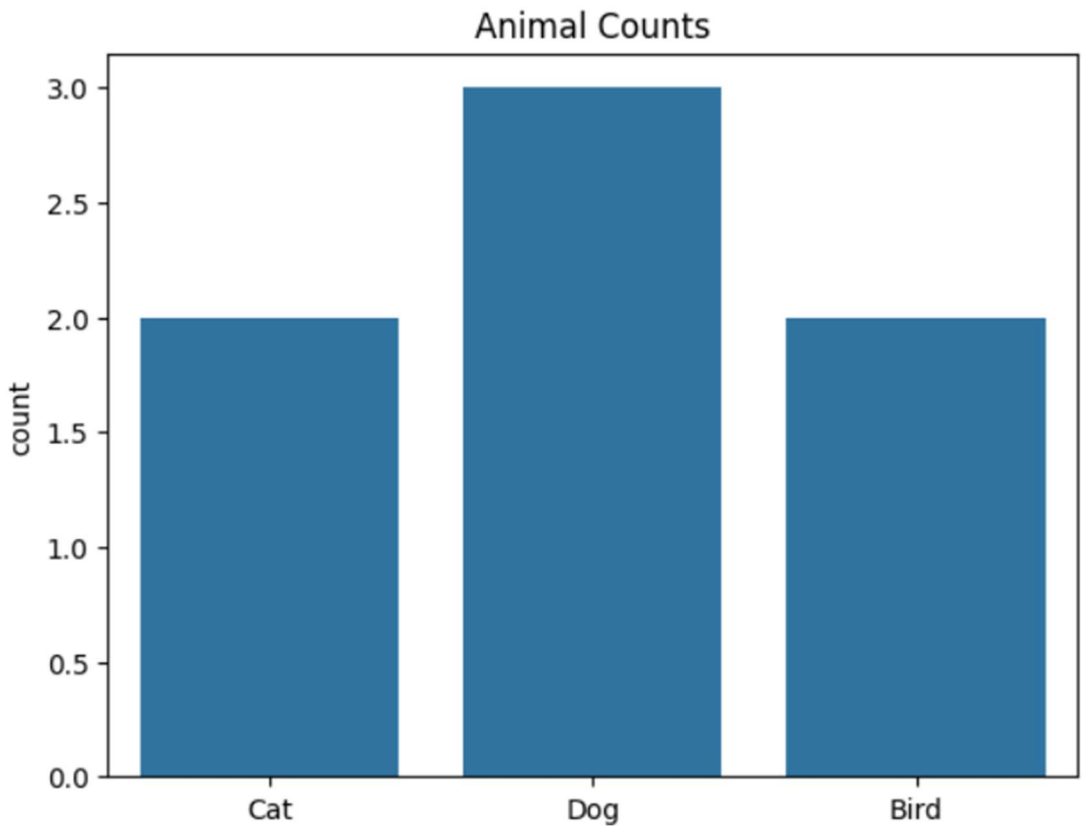
Bar plots are useful for comparing numerical values between categories.

SEABORN COUNT PLOT

A count plot displays how many observations belong to each category.

```
animals = ["Cat", "Dog", "Dog", "Bird", "Cat", "Dog", "Bird"]
sns.countplot(x=animals)
plt.title("Animal Counts")
plt.show()
```

Output



Explanation

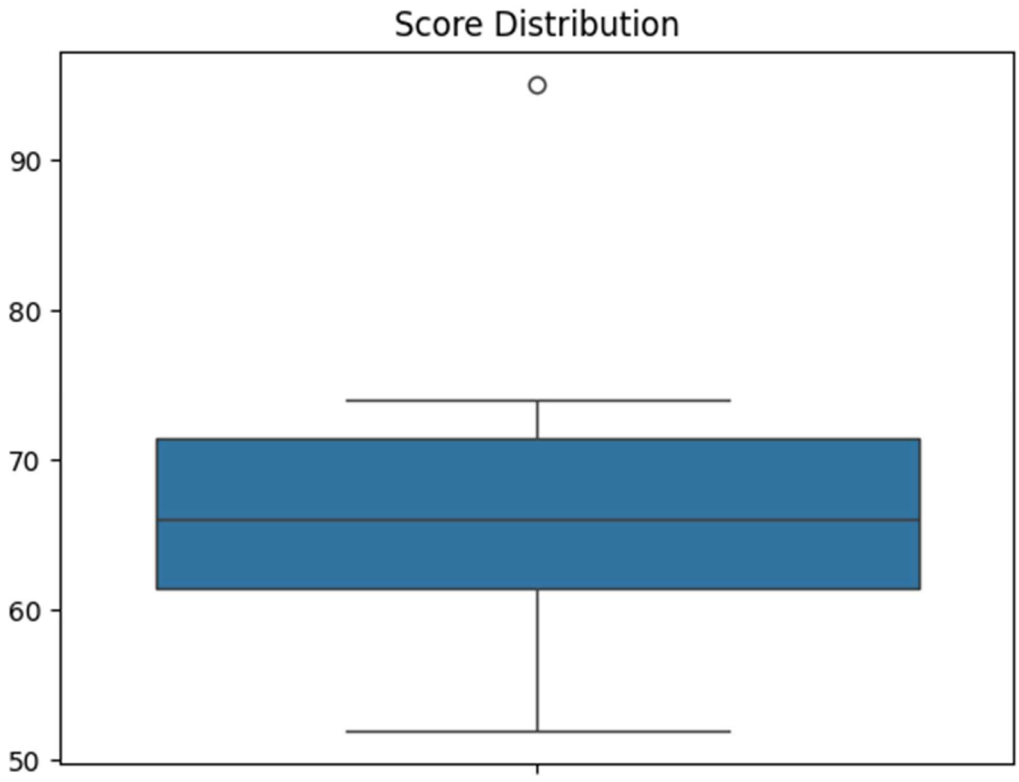
The height of each bar represents the number of records in that category.

BOX PLOT

A box plot summarizes the distribution of numerical data using quartiles. Values that fall far beyond the whiskers may appear as outliers.

```
scores = [52, 58, 61, 63, 65, 67, 70, 72, 74, 95]
sns.boxplot(y=scores)
plt.title("Score Distribution")
plt.show()
```

Output



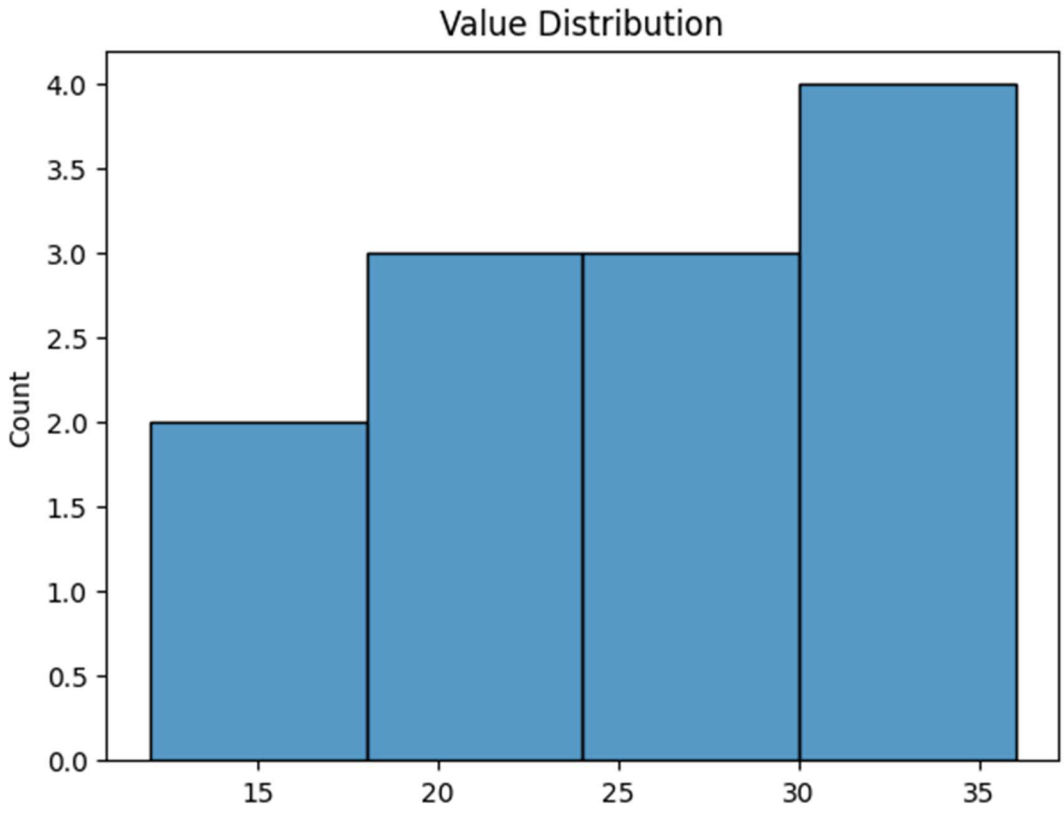
Explanation

The box represents the central portion of the data, while the whiskers extend toward the remaining typical observations.

HISTPLOT

```
values = [12, 15, 18, 19, 22, 24, 25, 27, 30, 31, 34, 36]
sns.histplot(values, bins=4)
plt.title("Value Distribution")
plt.show()
```

Output



Explanation

Histplot() groups numerical values into bins and displays their frequencies.

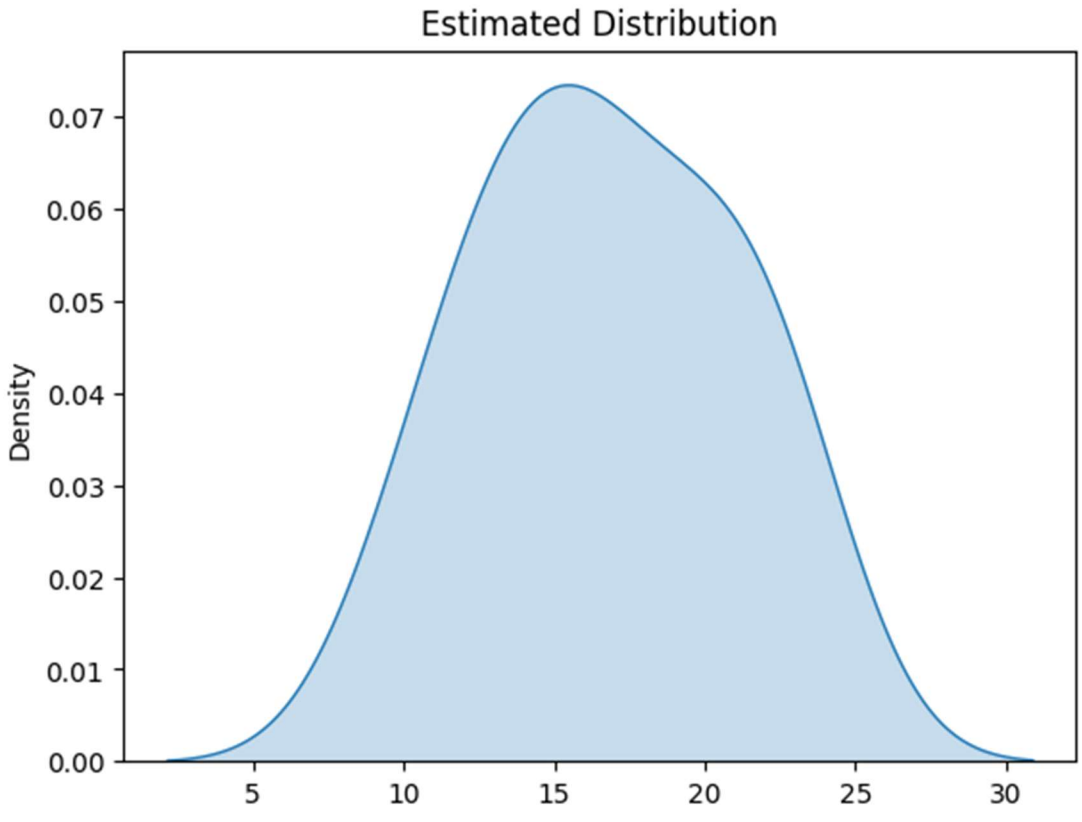
KDEPLOT

Explanation

KDE stands for Kernel Density Estimate. It draws a smooth curve that represents the estimated distribution of numerical observations.

```
values = [10, 12, 13, 15, 15, 16, 18, 19, 21, 22, 23]
sns.kdeplot(values, fill=True)
plt.title("Estimated Distribution")
plt.show()
```

Output



HEATMAP

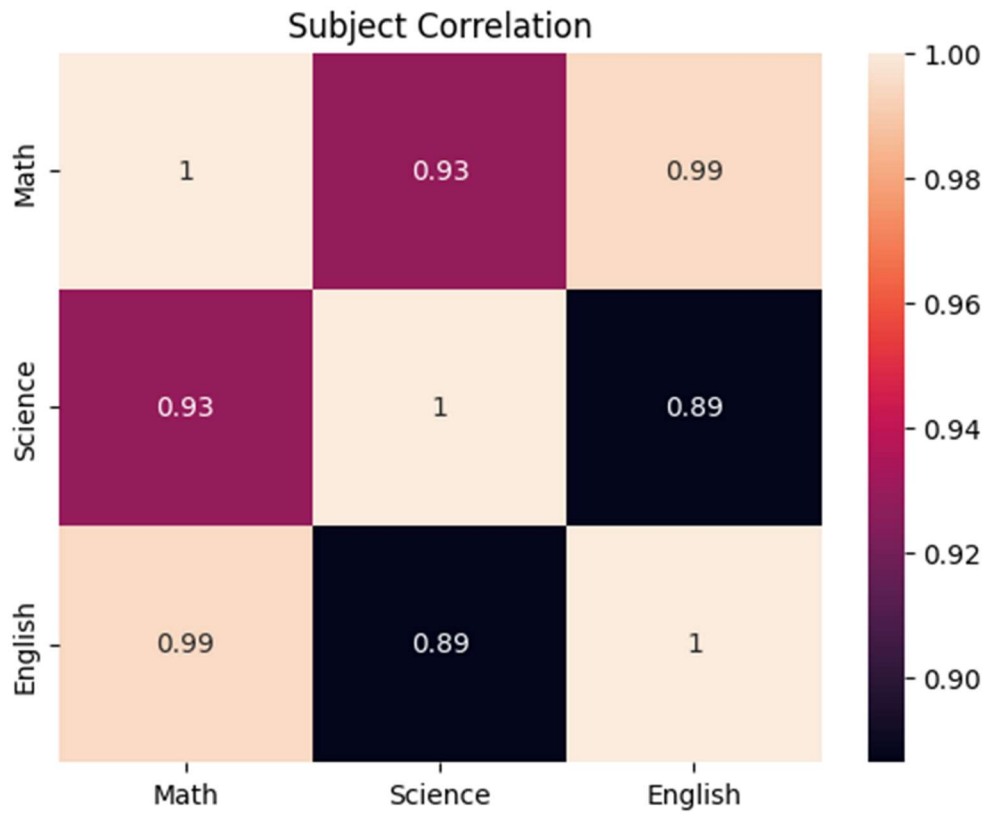
A heatmap represents values in a matrix using differences in visual intensity. It is often used to inspect correlations between numerical variables.

```
import pandas as pd

df = pd.DataFrame({
    "Math": [70, 82, 65, 91],
    "Science": [68, 79, 72, 88],
    "English": [75, 84, 69, 90]
})

sns.heatmap(df.corr(), annot=True)
plt.title("Subject Correlation")
plt.show()
```

Output



Explanation

The annot parameter displays the numerical correlation values inside the cells.

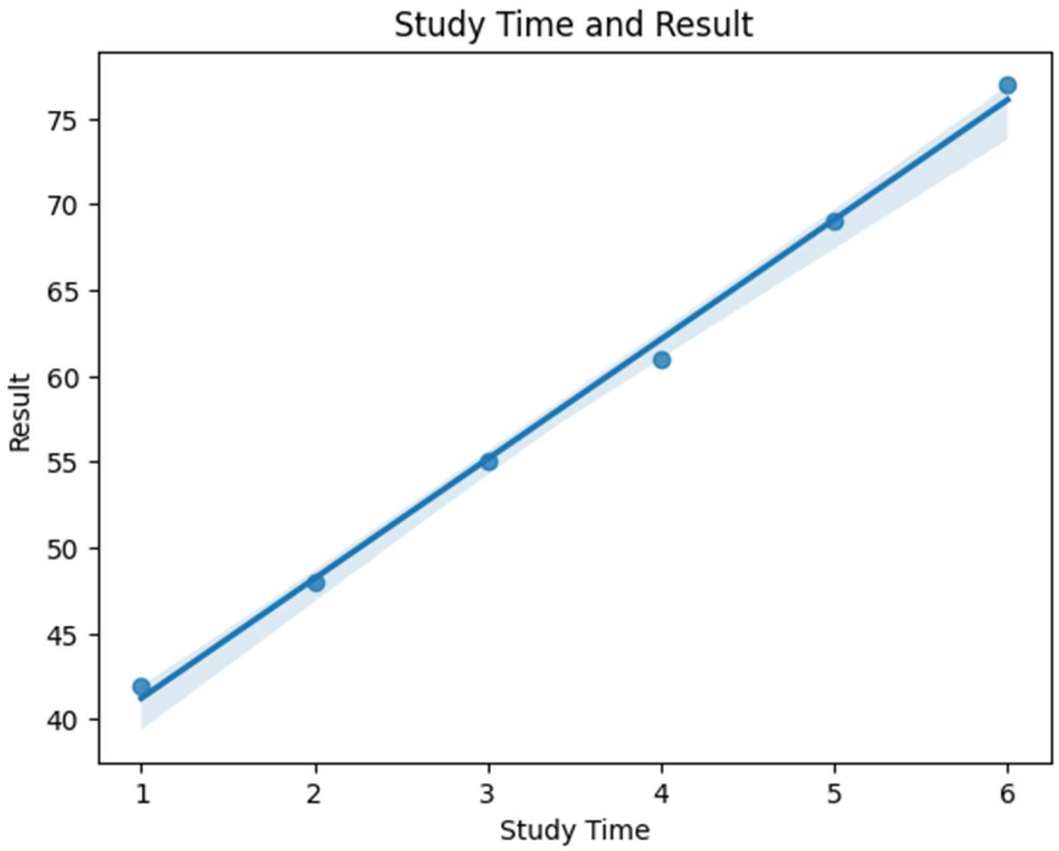
REGPLOT

A regression plot shows the relationship between two numerical variables and adds a fitted regression line.

```
study = [1, 2, 3, 4, 5, 6]
result = [42, 48, 55, 61, 69, 77]

sns.regplot(x=study, y=result)
plt.title("Study Time and Result")
plt.xlabel("Study Time")
plt.ylabel("Result")
plt.show()                          ,
```

Output



Explanation

The regression line provides a simple visual indication of the direction of the relationship.

COMPARISON OF MATPLOTLIB AND SEABORN

Matplotlib provides detailed control over individual plot elements and is suitable when precise customization is required.

- Highly customizable plotting system.
- Supports many basic and advanced chart types.
- Works well with NumPy, Pandas, and other Python tools.
- Useful when exact control over figure elements is needed.

Seaborn provides a higher-level interface focused on statistical and exploratory visualization.

- Simplifies the creation of statistical graphics.
- Provides convenient plotting functions.
- Includes themes and color palettes.
- Works smoothly with Pandas dataframes.
- Useful for exploratory data analysis.

In practice, both libraries can be used together. Matplotlib is useful for fine control, while Seaborn can make statistical visualization faster and more convenient.